Plan: Per-Vertex Top-m ABB Improvements (Global-Min, Multi-Criteria, Smaller-m, Fullness Gating)

2026-05-11

Background

The 2026-05-10 "v1" per-vertex ABB (`enumerate_top_k_per_vertex_cycles_signed_filtered_bb_batched_rs`) uses the *start vertex's* heap min as the upper-bound threshold. On Bitcoin Alpha and OTC seed-0 benches this gave 4.78× Rust walltime savings but lost −1.9 pp / −7.6 pp test AUC — cycles useful for some other cycle vertex's heap were thrown away when their score upper bound fell below the start vertex's threshold.

User directive (2026-05-11): pursue four orthogonal improvement axes labelled 1, 2, 4, 5.

Scope

- **Approach 1 – Global-min ABB.** Threshold is the minimum heap-min across all FULL vertex heaps, shared across rayon tasks via `AtomicU64` (encoding `f64` bits). Monotonically non-increasing; AUC-preserving by construction.
- **Approach 2 – Multi-criteria composite ABB.** Extend `BoundedScorer` to a `MultiBoundedScorer` that combines score upper bound with axiomatic upper bounds (balance, Davis-weak, geometric Ricci). Design only this round.
- **Approach 4 – Smaller m_v sweep.** Config-only: $m \in \{32, 64, 128\}$ on Bitcoin to test whether the correctness ceiling on small-data is reached at $m<128$.
- **Approach 5 – Adaptive fullness gating.** ABB only fires once a configurable fraction of vertex heaps are at capacity; defaults to 25%. Sweep $\in \{0.0, 0.1, 0.25, 0.5\}$.

Affected files

- `hymeko_graph/src/topk_cycles.rs` — `dfs_per_vertex_bb_global`, `atomic_min_f64`, `enumerate_top_k_per_vertex_c` plus three unit tests.
- `hymeko_graph/src/lib.rs` — export.
- `hymeko_py/src/cycles.rs` — `enumerate_top_k_per_vertex_cycles_signed_filtered_bb_global_batched_rs`.
- `hymeko_py/src/lib.rs` — registration.
- `signedkan_wip/src/n_tuples.py` — dispatch via `HSIKAN_USE_PER_VERTEX_ABB_MODE=global` and `HSIKAN_PER_VERTEX_` env vars.

CORE.YAML items touched

None. All edits land in the per-vertex enumerator, its PyO3 binding, and the Python dispatch shim. The signed-incidence math kernels, HSIKAN module API, and codegen templates remain untouched.

Interface changes

New public Rust function:

```
pub fn enumerate_top_k_per_vertex_cycles_par_adaptive_starting_bb_global_batched<P, S>(
    graph: &SignedGraph,
    k_len: usize,
    pruner: &P,
    m_v: &[u32],
    starting_vertices: &[u32],
    scorer: &S,
    fullness_gate: f64, // [0,1]; 1.0 effectively disables ABB
) -> TopKCyclesBatch
where P: CyclePruner + Sync, S: BoundedScorer + Sync;
```

New Python binding wraps it; output schema unchanged (cycles as `(N,k) u32`, scores as `(N,) f64`).

Test strategy

Three Rust unit tests, all on toy fixtures with ≤ 8 nodes:

1. `per_vertex_bb_global_huge_caps_matches_non_bb` — caps so large that no heap fills; ABB never fires; output count equals non-ABB.
2. `per_vertex_bb_global_subset_of_non_bb` — tight caps + dense graph; ABB fires; output is a SUBSET of non-ABB (correctness postcondition).
3. `per_vertex_bb_global_with_full_gate_disables_abb` — `fullness_gate=1.0`; ABB never activates; output matches non-ABB.

Python smoke test: parity of `(cycles, scores)` between the global-min binding at huge caps and the non-ABB binding.

Integration test on real data: Bitcoin Alpha + OTC seed-0 single-pass AUC compared against the ABB-OFF baseline; tolerance $\leq |0.005|$ for "AUC-preserving" verdict.

Performance budget

- Memory: bound by per-vertex heaps. With m_v=128, $k_{\max}$=6, `HeapEntry` $\approx$ 88 B, n=50,000 vertices: $\leq$ 0.6 GB. Within 16 GB cap.
- Wall-time (enumeration only): expect $\geq 1.5\times$ savings over non-ABB at gate= 0.25, $\leq 1.5\times$ slowdown vs v1 start-ABB. Slashdot k=4 canonical workload measures this.
- Inference latency unchanged (no model-architecture edit).

Rollback path

All env vars default to OFF / start-mode. `HSIKAN_USE_PER_VERTEX_ABB=0` reverts to the non-ABB filtered enumerator. The new PyO3 binding is additive — existing callers reach the same code path as before.

Protocol note

Implementation preceded this plan document due to the user's auto mode and a continuation-from-previous-context. The plan is filed retroactively to keep the four-artifact record (`plan.tex`, `plan.pdf`, `plan.tikz`, `plan.mmd`) intact for this change. The 2026-05-10 sister plan `docs/plans/2026-05-10-abb-global-topk/` covers global (*not* per-vertex) top-K ABB.